Todo Application — System Design (v1)

Version: 1.0
Date: 2026-03-01
Stack: Python 3.12, Django 5.x, Django REST Framework, PostgreSQL 16
Auth: Yo'q (v1)

---

1. Architecture

```
Client (Frontend)
    │
    │ HTTP / JSON
    ▼
┌──────────────────────────────────┐
│        Django + DRF              │
│                                  │
│  URLs → Views → Serializers      │
│            │          │          │
│          Models (ORM)            │
│            │          │          │
│         PostgreSQL               │
└──────────────────────────────────┘
```

Layerlar:

- **URLs** — endpoint routing
- **Views** — `ModelViewSet` orqali CRUD + custom actions
- **Serializers** — validation, request/response transformation
- **Models** — ORM, business logic, constraints
- **Managers** — custom querysetlar (filter, annotate)

---

2. Django Project Structure

```
todo_project/
├── config/
│   ├── settings/
│   │   ├── base.py
```

```
│   │   ├── local.py
│   │   └── production.py
│   ├── urls.py
│   └── wsgi.py
├── apps/
│   ├── folders/
│   │   ├── models.py
│   │   ├── serializers.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── admin.py
│   │   └── tests/
│   ├── categories/
│   │   ├── models.py
│   │   ├── serializers.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── admin.py
│   │   └── tests/
│   ├── todos/
│   │   ├── models.py
│   │   ├── serializers.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── filters.py
│   │   ├── admin.py
│   │   └── tests/
│   ├── tags/
│   │   ├── models.py
│   │   ├── serializers.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── admin.py
│   │   └── tests/
│   ├── labels/
│   │   ├── models.py
│   │   ├── serializers.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── admin.py
│   │   └── tests/
│   └── common/
│       ├── models.py        ← TimeStampedModel (abstract)
│       ├── pagination.py
```

```
│       └── exceptions.py
├── manage.py
├── requirements/
│   ├── base.txt
│   ├── local.txt
│   └── production.txt
├── docker-compose.yml
├── Dockerfile
└── .env
```

Best Practices:

- Har bir entity alohida Django app
- Settings split (base / local / production)
- `common` app — shared abstract modellar, pagination, exception handler
- `tests/` har bir app ichida

---

3. Database Schema

3.1 ERD

```
┌──────────────────┐      ┌──────────────────┐
│   folders        │      │   categories     │
├──────────────────┤      ├──────────────────┤
│ id (UUID)        │      │ id (UUID)        │
│ name             │      │ name (unique)    │
│ color            │      │ color            │
│ icon             │      │ icon             │
│ position         │      │ created_at       │
│ created_at       │      │ updated_at       │
│ updated_at       │      └──────────────────┘
└──────────────────┘                │
        │ 1:N                       │ 1:N (nullable)
        ▼                           ▼
┌──────────────────────────────────────────────┐
│                    todos                     │
├──────────────────────────────────────────────┤
│ id (UUID)                                    │
│ folder      → FK folders (CASCADE)           │
│ category    → FK categories (SET_NULL)       │
│ title                                        │
```

```
│ description                    │
│ priority   (low/medium/high/urgent)   │
│ status     (pending/in_progress/done/cancelled) │
│ due_date                       │
│ position                       │
│ created_at                     │
│ updated_at                     │
└──────────────────────────────────────────────────┘
    │      │          │
    │      │          │
    │1:N   │M2M       │M2M
    ▼      ▼          ▼

┌──────────┐  ┌──────────┐  ┌──────────┐
│sub       │  │tags      │  │labels    │
│todos     │  │          │  │          │
├──────────┤  ├──────────┤  ├──────────┤
│id        │  │id        │  │id        │
│todo→     │  │name      │  │name      │
│title     │  │          │  │color     │
│is_done   │  │          │  │          │
│position  │  │          │  │          │
└──────────┘  └──────────┘  └──────────┘
```

3.2 Table Definitions

TimeStampedModel (abstract — barcha modellar meros oladi)

Field	Type	Note
id	UUIDField	PK, auto-generated
created_at	DateTimeField	auto_now_add=True
updated_at	DateTimeField	auto_now=True

---

folders

Field	Type	Constraints
id	UUID	PK
name	CharField(255)	required
color	CharField(7)	default="#6366F1"
icon	CharField(50)	default="folder"
position	PositiveIntegerField	default=0
created_at	DateTimeField	auto
updated_at	DateTimeField	auto

Ordering: position, created_at
Index: position

---

categories

Field	Type	Constraints
id	UUID	PK
name	CharField(255)	required, unique
color	CharField(7)	default="#8B5CF6"
icon	CharField(50)	default="tag"
created_at	DateTimeField	auto
updated_at	DateTimeField	auto

---

todos

Field	Type	Constraints
id	UUID	PK
folder	ForeignKey(Folder)	on_delete=CASCADE, related_name="todos"
category	ForeignKey(Category)	on_delete=SET_NULL, null=True, blank=True
title	CharField(500)	required
description	TextField	blank=True, default=""
priority	CharField(10)	choices: low / medium / high / urgent, default="medium"
status	CharField(15)	choices: pending / in_progress / done / cancelled, default="pending"
due_date	DateTimeField	null=True, blank=True
position	PositiveIntegerField	default=0
tags	ManyToManyField(Tag)	blank=True, related_name="todos"
labels	ManyToManyField(Label)	blank=True, related_name="todos"
created_at	DateTimeField	auto
updated_at	DateTimeField	auto

Indexes: `folder`, `category`, `status`, `priority`, `due_date`, (`folder`, `position`)
Ordering: `position`, `-created_at`

> **M2M note:** Django avtomatik `todo_tags` va `todo_labels` junction tablelarni yaratadi. Agar keyinchalik qo'shimcha fieldlar kerak bo'lsa (masalan, `assigned_at`), `through` model ishlatiladi.

---

sub_todos

Field	Type	Constraints
id	UUID	PK
todo	ForeignKey(Todo)	on_delete=CASCADE, related_name="sub_todos"
title	CharField(500)	required
is_completed	BooleanField	default=False
position	PositiveIntegerField	default=0
created_at	DateTimeField	auto
updated_at	DateTimeField	auto

Indexes: todo, (todo, position)
Ordering: position

---

tags

Field	Type	Constraints
id	UUID	PK
name	CharField(100)	required, unique, db_index=True
created_at	DateTimeField	auto
updated_at	DateTimeField	auto

Note: Rangsiz. Faqat text-based filterlash uchun. (#api, #bugfix, #v2)

---

labels

Field	Type	Constraints
id	UUID	PK
name	CharField(100)	required, unique, db_index=True
color	CharField(7)	default="`#6B7280`"
created_at	DateTimeField	auto
updated_at	DateTimeField	auto

Note: Rangli vizual markerlar. (🔴 Critical, 🟡 In Review, 🟢 Ready)

3.3 Tags vs Labels

	Tags	Labels
Maqsad	Erkin qidiruv, filterlash	Vizual status/kategoriya
Rang	❌	✅
Misol	`#api`, `#bugfix`, `#v2`	Critical, In Review, Ready
Relation	M2M	M2M

---

4. API Design

Base URL: `/api/v1/`
Format: JSON
Pagination: `PageNumberPagination` (default: 20, max: 100)
Filtering: `django-filter` + custom FilterSet

4.1 Standard Response Format

Success:

```
json
```

```
{
  "count": 10,
  "next": "http://localhost:8000/api/v1/todos/?page=2",
  "previous": null,
  "results": [ ... ]
}
```

Single object (create/update/detail):

```json
{
  "id": "uuid",
  "name": "...",
  ...
}
```

Error (400):

```json
{
  "field_name": ["Error message."],
  "non_field_errors": ["General error."]
}
```

Error (404):

```json
{
  "detail": "Not found."
}
```

> **Note:** DRF standart response formati ishlatiladi. Custom wrapper qo'shilmaydi — bu DRF best practice.

4.2 Folders

#	Method	Endpoint	Description
1	GET	`/folders/`	List all folders

#	Method	Endpoint	Description
2	POST	/folders/	Create folder
3	GET	/folders/{id}/	Get folder
4	PUT	/folders/{id}/	Full update
5	PATCH	/folders/{id}/	Partial update
6	DELETE	/folders/{id}/	Delete folder (cascade)

POST /folders/

Request:

```json
{
  "name": "Work Projects",
  "color": "#6366F1",
  "icon": "briefcase",
  "position": 0
}
```

Response (201):

```json
{
  "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "name": "Work Projects",
  "color": "#6366F1",
  "icon": "briefcase",
  "position": 0,
  "todo_count": 0,
  "created_at": "2026-03-01T10:00:00Z",
  "updated_at": "2026-03-01T10:00:00Z"
}
```

GET /folders/

Response (200):

```json
```

```
[
  {
    "id": "a1b2c3d4-...",
    "name": "Work Projects",
    "color": "#6366F1",
    "icon": "briefcase",
    "position": 0,
    "todo_count": 5,
    "created_at": "2026-03-01T10:00:00Z",
    "updated_at": "2026-03-01T10:00:00Z"
  },
  {
    "id": "b2c3d4e5-...",
    "name": "Personal",
    "color": "#EC4899",
    "icon": "heart",
    "position": 1,
    "todo_count": 3,
    "created_at": "2026-03-01T10:05:00Z",
    "updated_at": "2026-03-01T10:05:00Z"
  }
]
```

> **Note:** Folders paginate qilinmaydi (kam sonli). List response `[]` array.

4.3 Categories

#	Method	Endpoint	Description
7	GET	`/categories/`	List all
8	POST	`/categories/`	Create
9	GET	`/categories/{id}/`	Detail
10	PUT	`/categories/{id}/`	Full update
11	PATCH	`/categories/{id}/`	Partial update
12	DELETE	`/categories/{id}/`	Delete

POST /categories/

Request:

```json
{
  "name": "Backend",
  "color": "#10B981",
  "icon": "server"
}
```

Response (201):

```json
{
  "id": "c3d4e5f6-...",
  "name": "Backend",
  "color": "#10B981",
  "icon": "server",
  "todo_count": 0,
  "created_at": "2026-03-01T10:00:00Z",
  "updated_at": "2026-03-01T10:00:00Z"
}
```

4.4 Tags

#	Method	Endpoint	Description
13	GET	`/tags/`	List all
14	POST	`/tags/`	Create
15	GET	`/tags/{id}/`	Detail
16	PUT	`/tags/{id}/`	Full update
17	PATCH	`/tags/{id}/`	Partial update
18	DELETE	`/tags/{id}/`	Delete

POST /tags/

Request:

```json
{
  "name": "api"
}
```

Response (201):

```json
{
  "id": "t1a2b3c4-...",
  "name": "api",
  "todo_count": 0,
  "created_at": "2026-03-01T10:00:00Z",
  "updated_at": "2026-03-01T10:00:00Z"
}
```

4.5 Labels

#	Method	Endpoint	Description
19	GET	`/labels/`	List all
20	POST	`/labels/`	Create
21	GET	`/labels/{id}/`	Detail
22	PUT	`/labels/{id}/`	Full update
23	PATCH	`/labels/{id}/`	Partial update
24	DELETE	`/labels/{id}/`	Delete

POST /labels/

Request:

```json
```

```
{
  "name": "Critical",
  "color": "#EF4444"
}
```

Response (201):

```json
{
  "id": "l1a2b3c4-...",
  "name": "Critical",
  "color": "#EF4444",
  "todo_count": 0,
  "created_at": "2026-03-01T10:00:00Z",
  "updated_at": "2026-03-01T10:00:00Z"
}
```

4.6 Todos

#	Method	Endpoint	Description
25	GET	`/todos/`	List (filtered, paginated)
26	POST	`/todos/`	Create
27	GET	`/todos/{id}/`	Detail (nested sub_todos, tags, labels)
28	PUT	`/todos/{id}/`	Full update
29	PATCH	`/todos/{id}/`	Partial update
30	DELETE	`/todos/{id}/`	Delete (cascade sub_todos)

Filter Parameters (GET /todos/)

Param	Type	Example	Description
folder	UUID	`?folder=uuid`	Filter by folder
category	UUID	`?category=uuid`	Filter by category
status	string	`?status=pending`	Filter by status
priority	string	`?priority=high`	Filter by priority
tags	UUID	`?tags=uuid1,uuid2`	Filter by tags (AND)
labels	UUID	`?labels=uuid1,uuid2`	Filter by labels (AND)
due_date_before	datetime	`?due_date_before=2026-03-15`	Due before date
due_date_after	datetime	`?due_date_after=2026-03-01`	Due after date
search	string	`?search=api`	Search title & description
ordering	string	`?ordering=-due_date`	Sort field (prefix `-` for desc)
page	int	`?page=2`	Page number

POST /todos/

Request:

```json
{
  "folder": "a1b2c3d4-...",
  "category": "c3d4e5f6-...",
  "title": "Implement REST API",
  "description": "CRUD endpoints with validation",
  "priority": "high",
  "due_date": "2026-03-15T18:00:00Z",
  "tag_ids": ["t1a2b3c4-...", "t2b3c4d5-..."],
  "label_ids": ["l1a2b3c4-..."]
}
```

Response (201):

```json
```

```
{
  "id": "e5f6a7b8-...",
  "folder": "a1b2c3d4-...",
  "category": "c3d4e5f6-...",
  "title": "Implement REST API",
  "description": "CRUD endpoints with validation",
  "priority": "high",
  "status": "pending",
  "due_date": "2026-03-15T18:00:00Z",
  "position": 0,
  "tags": [
    { "id": "t1a2b3c4-...", "name": "api" },
    { "id": "t2b3c4d5-...", "name": "bugfix" }
  ],
  "labels": [
    { "id": "l1a2b3c4-...", "name": "Critical", "color": "#EF4444" }
  ],
  "sub_todo_stats": {
    "total": 0,
    "completed": 0
  },
  "created_at": "2026-03-01T10:30:00Z",
  "updated_at": "2026-03-01T10:30:00Z"
}
```

GET /todos/ — List (paginated)

Response (200):

```
json
```

```
{
  "count": 42,
  "next": "http://localhost:8000/api/v1/todos/?page=2",
  "previous": null,
  "results": [
    {
      "id": "e5f6a7b8-...",
      "folder": "a1b2c3d4-...",
      "category": "c3d4e5f6-...",
      "title": "Implement REST API",
      "description": "CRUD endpoints with validation",
      "priority": "high",
      "status": "pending",
      "due_date": "2026-03-15T18:00:00Z",
      "position": 0,
      "tags": [
        { "id": "t1a2b3c4-...", "name": "api" },
        { "id": "t2b3c4d5-...", "name": "bugfix" }
      ],
      "labels": [
        { "id": "l1a2b3c4-...", "name": "Critical", "color": "#EF4444" }
      ],
      "sub_todo_stats": {
        "total": 3,
        "completed": 1
      },
      "created_at": "2026-03-01T10:30:00Z",
      "updated_at": "2026-03-01T10:30:00Z"
    }
  ]
}
```

GET /todos/{id}/ — Detail

Response (200):

```
json
```

```
{
  "id": "e5f6a7b8-...",
  "folder": {
    "id": "a1b2c3d4-...",
    "name": "Work Projects",
    "color": "#6366F1"
  },
  "category": {
    "id": "c3d4e5f6-...",
    "name": "Backend",
    "color": "#10B981"
  },
  "title": "Implement REST API",
  "description": "CRUD endpoints with validation",
  "priority": "high",
  "status": "pending",
  "due_date": "2026-03-15T18:00:00Z",
  "position": 0,
  "tags": [
    { "id": "t1a2b3c4-...", "name": "api" },
    { "id": "t2b3c4d5-...", "name": "bugfix" }
  ],
  "labels": [
    { "id": "l1a2b3c4-...", "name": "Critical", "color": "#EF4444" }
  ],
  "sub_todos": [
    {
      "id": "f6a7b8c9-...",
      "title": "Define DTOs",
      "is_completed": true,
      "position": 0
    },
    {
      "id": "a7b8c9d0-...",
      "title": "Implement handlers",
      "is_completed": false,
      "position": 1
    },
    {
      "id": "b8c9d0e1-...",
      "title": "Write tests",
      "is_completed": false,
      "position": 2
```

```
    }
  ],
  "sub_todo_stats": {
    "total": 3,
    "completed": 1
  },
  "created_at": "2026-03-01T10:30:00Z",
  "updated_at": "2026-03-01T10:30:00Z"
}
```

> **Serializer strategy:** List uchun `TodoListSerializer` (folder/category faqat UUID), Detail uchun `TodoDetailSerializer` (nested objects + sub_todos).

PATCH /todos/{id}/ — Partial Update

Request (masalan faqat status o'zgartirish):

```json
{
  "status": "in_progress"
}
```

Response (200):

```json
{
  "id": "e5f6a7b8-...",
  "folder": "a1b2c3d4-...",
  "category": "c3d4e5f6-...",
  "title": "Implement REST API",
  "description": "CRUD endpoints with validation",
  "priority": "high",
  "status": "in_progress",
  "due_date": "2026-03-15T18:00:00Z",
  "position": 0,
  "tags": [...],
  "labels": [...],
  "sub_todo_stats": { "total": 3, "completed": 1 },
  "created_at": "2026-03-01T10:30:00Z",
  "updated_at": "2026-03-01T12:00:00Z"
}
```

> **Note:** DRF `ModelViewSet` da `PUT` = full update, `PATCH` = partial update. Alohida `/status` endpoint kerak emas — `PATCH` yetarli.

4.7 Sub-Todos

#	Method	Endpoint	Description
31	GET	`/todos/{todo_id}/sub-todos/`	List sub-todos
32	POST	`/todos/{todo_id}/sub-todos/`	Create sub-todo
33	GET	`/todos/{todo_id}/sub-todos/{id}/`	Detail
34	PUT	`/todos/{todo_id}/sub-todos/{id}/`	Full update
35	PATCH	`/todos/{todo_id}/sub-todos/{id}/`	Partial update
36	DELETE	`/todos/{todo_id}/sub-todos/{id}/`	Delete

> **Nested router** (`drf-nested-routers`) orqali: sub-todo har doim todo kontekstida.

POST /todos/{todo_id}/sub-todos/

Request:

```json
{
  "title": "Write unit tests",
  "position": 0
}
```

Response (201):

```json
```

```
{
  "id": "c9d0e1f2-...",
  "todo": "e5f6a7b8-...",
  "title": "Write unit tests",
  "is_completed": false,
  "position": 0,
  "created_at": "2026-03-01T13:00:00Z",
  "updated_at": "2026-03-01T13:00:00Z"
}
```

PATCH /todos/{todo_id}/sub-todos/{id}/ — Toggle

Request:

```
json

{
  "is_completed": true
}
```

Response (200):

```
json

{
  "id": "c9d0e1f2-...",
  "todo": "e5f6a7b8-...",
  "title": "Write unit tests",
  "is_completed": true,
  "position": 0,
  "created_at": "2026-03-01T13:00:00Z",
  "updated_at": "2026-03-01T14:00:00Z"
}
```

---

5. Error Responses

Validation Error (400)

```
json
```

```
{
  "title": ["This field is required."],
  "folder": ["Invalid pk \"xyz\" - object does not exist."]
}
```

Not Found (404)

```json
{
  "detail": "Not found."
}
```

Unique Constraint (400)

```json
{
  "name": ["Tag with this name already exists."]
}
```

Method Not Allowed (405)

```json
{
  "detail": "Method \"DELETE\" not allowed."
}
```

---

6. Full API Summary

#	Method	Endpoint	Description
1	GET	`/folders/`	List folders
2	POST	`/folders/`	Create folder
3	GET	`/folders/{id}/`	Folder detail
4	PUT	`/folders/{id}/`	Update folder

#	Method	Endpoint	Description
5	PATCH	/folders/{id}/	Partial update folder
6	DELETE	/folders/{id}/	Delete folder
7	GET	/categories/	List categories
8	POST	/categories/	Create category
9	GET	/categories/{id}/	Category detail
10	PUT	/categories/{id}/	Update category
11	PATCH	/categories/{id}/	Partial update category
12	DELETE	/categories/{id}/	Delete category
13	GET	/tags/	List tags
14	POST	/tags/	Create tag
15	GET	/tags/{id}/	Tag detail
16	PUT	/tags/{id}/	Update tag
17	PATCH	/tags/{id}/	Partial update tag
18	DELETE	/tags/{id}/	Delete tag
19	GET	/labels/	List labels
20	POST	/labels/	Create label
21	GET	/labels/{id}/	Label detail
22	PUT	/labels/{id}/	Update label
23	PATCH	/labels/{id}/	Partial update label
24	DELETE	/labels/{id}/	Delete label
25	GET	/todos/	List todos (filtered, paginated)
26	POST	/todos/	Create todo
27	GET	/todos/{id}/	Todo detail (nested)

#	Method	Endpoint	Description
28	PUT	/todos/{id}/	Update todo
29	PATCH	/todos/{id}/	Partial update todo
30	DELETE	/todos/{id}/	Delete todo
31	GET	/todos/{todo_id}/sub-todos/	List sub-todos
32	POST	/todos/{todo_id}/sub-todos/	Create sub-todo
33	GET	/todos/{todo_id}/sub-todos/{id}/	Sub-todo detail
34	PUT	/todos/{todo_id}/sub-todos/{id}/	Update sub-todo
35	PATCH	/todos/{todo_id}/sub-todos/{id}/	Partial update sub-todo
36	DELETE	/todos/{todo_id}/sub-todos/{id}/	Delete sub-todo

Total: 36 endpoints

---

7. Key Dependencies

Package	Purpose
Django 5.x	Framework
djangorestframework	REST API
django-filter	Filterlash
drf-nested-routers	Sub-todo nested URLs
psycopg[binary]	PostgreSQL driver
django-cors-headers	CORS
drf-spectacular	OpenAPI/Swagger docs
python-decouple	Environment variables
gunicorn	Production server

---

8. Best Practices (v1)

Practice	Detail
UUID primary keys	Predictable ID exposure yo'q
Abstract TimeStampedModel	DRY — barcha modellar meros oladi
Separate serializers	List (flat) vs Detail (nested)
django-filter FilterSet	Query param filterlash — viewdan ajratilgan
Nested routers	Sub-todo faqat todo kontekstida accessible
select_related / prefetch_related	N+1 query muammosi oldini olish
Trailing slash	DRF standart — `/todos/` not `/todos`
PATCH for partial	Alohida status endpoint o'rniga standart PATCH
M2M via serializer	Tag/label attach-detach create/update da `tag_ids` / `label_ids` orqali
Pagination faqat kerakli joyda	Todos — paginated; Folders, Categories, Tags, Labels — unpaginated
drf-spectacular	Auto-generated Swagger/OpenAPI documentation